Runbook détaillé — Solution A

FX pur · Convention A · NautilusTrader · Backtest · Latence

Document interne · Niveau formel · Octobre 2026

1. Objectif et stade actuel

Ce runbook formalise la trajectoire complète de la Solution A : univers FX pur (8 devises cœur, 28 paires), Convention A (marquage aux prix exécutables), jusqu'à l'intégration dans NautilusTrader, le backtest interne sur données historiques déjà téléchargées, les tests de performance et de latence, puis la préparation de l'adapter venue. À ce stade, seul le cadre méthodologique est figé. Le code Rust et la Strategy NautilusTrader restent à implémenter. Le document constitue la référence unique pour éviter toute dérive de périmètre.

2. NautilusTrader — Architecture et intégration

NautilusTrader est une plateforme open-source (cœur Rust, interfaces Python et Rust) conçue selon le domain-driven design, l'architecture event-driven, le pattern ports-and-adapters (hexagonal) et le crash-only design. Elle garantit la parité recherche-to-live : le même NautilusKernel est partagé entre le BacktestEngine et le LiveNode. Seules les sources de données et les clients d'exécution diffèrent.

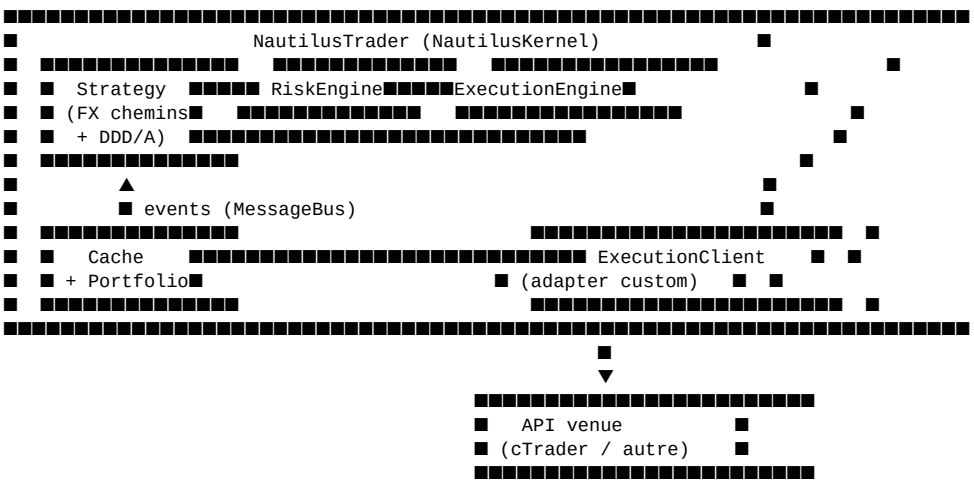
2.1 Composants centraux utilisés

NautilusKernel : orchestration centrale, cycle de vie du nœud, horloge, frontière unique.
MessageBus : routage pub/sub, request/response et point-to-point avec garantie d'ordre.
Cache : état in-memory indexé (instruments, ordres, positions, données de marché).
DataEngine : ingestion, normalisation et distribution des quotes et ticks.
ExecutionEngine : cycle de vie des ordres, routage vers les ExecutionClient, reconciliation.
RiskEngine : contrôles pré-trade (quantité, notionnel, marge, limites de taux).
Portfolio : positions nettes, PnL réalisé et latent, exposition, marge.
Trader : enregistrement et cycle de vie des Strategies et Actors.

2.2 Rôle de la Strategy

La logique algorithmique (énumération des chemins ouverts $L \leq 3$, calcul du DDD_eff sous Convention A, règle d'envoi) est portée dans une Strategy. Celle-ci reçoit les events de marché via le MessageBus, accède au Cache et au Portfolio, et émet des commandes d'ordre. Elle n'interagit jamais directement avec l'API venue.

2.3 Schéma d'intégration cible



3. Backtest interne

Dès que la Strategy est construite, le BacktestEngine de NautilusTrader permet d'exécuter exactement le même code sur les données historiques déjà téléchargées (campagne v2). Les données sont chargées via un DataClient historique ou un loader. Les fills sont simulés selon les règles configurées. Aucun adapter live n'est requis à cette étape. Cette phase valide le calcul des chemins, le DDD_eff et la règle d'envoi avant toute connexion à une API externe.

4. Adapter venue (cTrader ou autre)

NautilusTrader ne fournit pas d'adapter cTrader clé en main, mais le pattern ports-and-adapters le permet explicitement. Un package custom (Python ou Rust/PyO3) implémente les traits DataClient et ExecutionClient. L'ExecutionClient traduit les commandes Nautilus (submit, modify, cancel) en appels API venue, émet les events d'ordre et de fill, et génère les rapports de reconciliation (mass status au démarrage, probes périodiques). Le DataClient gère les souscriptions et la fraîcheur des events. L'adapter s'enregistre dans le LiveNode ; la Strategy reste inchangée.

5. Tests de performance et de latence

Ces tests sont obligatoires avant le holdout. Ils garantissent que le moteur respecte les contraintes temporelles d'un arbitrage FX à chemins ouverts.

5.1 Métriques cibles

• Latence de calcul d'un chemin (énumération/DP + DDD_eff) : < 1 ms en p99.
• Latence end-to-end Strategy → ordre émis (backtest) : < 5 ms en p99.
• Débit : ≥ 10 000 ticks/s en mode backtest sans dégradation.
• Empreinte mémoire stable sur une séance complète (absence de fuite).

5.2 Protocole

1. Instrumenter le cœur de calcul et la Strategy (timestamps nanosecondes).
2. Rejouer un extrait représentatif des données historiques.
3. Collecter les distributions p50 / p95 / p99 et le débit.
4. Comparer le mode backtest et le mode live simulé (même noyau).
5. Archiver les traces dans Vultr et les inclure dans le rapport PoC.
Tout dépassement des seuils bloque la suite jusqu'à optimisation.

6. Séquence d'exécution de bout en bout

Étape	Description
1	Créer l'Object Storage Vultr et la structure (raw/, processed/, holdout/, reports/, artifacts/)
2	Initialiser le dépôt GitHub et le squelette Rust
3	Implémenter le modèle de données et le cœur de calcul FX (Convention A, L ≤ 3)
4	Porter la logique en Strategy NautilusTrader
5	Charger les données historiques et exécuter le backtest interne
6	Exécuter les tests de performance et de latence
7	Isoler le jour 26 comme holdout et produire les métriques (PedAnova)
8	Rédiger le rapport PoC FX (PDF formel, incluant latence)
9	Déposer les artifacts et le rapport dans Vultr
10	Développer l'adapter DataClient / ExecutionClient (cTrader)
11	Push final sur GitHub + tag (code, tests, rapport, runbook, liens)

7. Points de contrôle

• Convention A exclusive sur les chemins FX purs.
• Aucune activation des bouts CFD avant validation du cœur et du rapport PoC.

• Parité backtest/live assurée par le NautilusKernel partagé.
• Tests de latence validés avant le holdout.
• L’adapter venue est le seul composant spécifique à l’API externe.
• Le dépôt GitHub final doit être reproductible et auditable de bout en bout.

Fin du runbook détaillé — Stade actuel : cadre méthodologique figé, implémentation à démarrer.